

Recursive Descent Agent — V2 Roadmap

Version Philosophy

V1 is a code auditor that knows about your business.

V2.0 is a technical co-founder that remembers, sees through multiple lenses, and ties code to revenue.

V2.1 is a product strategist that simulates users and studies competitors.

V2.2 is a learning system that gets smarter every week it runs.

Each sub-version is fully functional on its own. You ship 2.0, use it for a few weeks, then layer on 2.1 when ready.

V2.0 — "Persistent Memory + Multi-Lens Analysis"

The jump from V1: The agent stops re-reading the entire codebase every session, analyzes through three distinct expert lenses instead of one generic voice, ties every finding to business impact, and catches when your documentation drifts from your code.

What's new

1. Persistent Codebase Memory

What it does: On the first run, the agent builds `.agent/knowledge-graph.json` — a structured map of every file in the project: path, content hash, dependencies (imports/exports), descent level reached, scores per dimension, flags found, and last-analyzed date.

On every subsequent run, instead of re-traversing everything:

1. `git diff --name-only [last session commit hash]` → changed files
2. For each changed file → find all dependents in the knowledge graph
3. Re-descend **ONLY** into changed files + their dependents
4. Inherit prior scores for everything untouched
5. Update `knowledge-graph.json` with new hashes and scores

Why it matters: A full V1 cycle on a mid-size project takes 3–5 hours. With persistent memory, a typical week's changes trigger re-analysis of maybe 15–30 files instead of 300+. Weekly cycles drop to 30–60 minutes after the initial run. This alone makes daily usage realistic instead of aspirational.

Knowledge graph structure:

json

```
{
  "version": "2.0",
  "project": "field-advisor",
  "last_session": "2026-03-28",
  "last_commit": "abc123f",
  "files": {
    "/app/api/bookings/create/route.ts": {
      "hash": "sha256:...",
      "last_analyzed": "2026-03-28",
      "descent_level_reached": 3,
      "module": "booking",
      "dependencies": ["/lib/stripe/create-intent.ts", "/lib/business-rules/boo
      "dependents": ["/components/booking/BookingForm.tsx"],
      "scores": {
        "completeness": 6,
        "robustness": 4,
        "quality": 7,
        "security": 5,
        "test_coverage": 2
      },
      "flags": ["missing-idempotency", "no-zod-validation"],
      "persona_flags": {
        "architect": ["inline-business-logic"],
        "user": [],
        "adversary": ["no-rate-limiting", "session-only-auth"]
      }
    }
  },
  "modules": {
    "booking": {
      "files": [...],
      "overall_scores": {},
      "milestone_relevance": {"M1": "critical", "M2": "critical", "M3": "releva
      "last_full_descent": "2026-03-28"
    }
  },
  "cross_cut_patterns": {
    "missing-error-states": {
      "instance_count": 12,
      "files": [...],
      "last_detected": "2026-03-28"
    }
  }
}
```

Delta analysis protocol:

```
## Delta Analysis – [DATE]

### Files changed since last session: [n]
[list with change type: modified / added / deleted]

### Dependency cascade: [n] additional files to re-analyze
[list with reason: "depends on [changed file]"]

### Files skipped (unchanged, scores inherited): [n]

### Re-descent scope: [n] files across [n] modules
[This is your analysis surface for today – everything else keeps prior scores]
```

New session format — Delta Run (30–60 min):

```
Read .agent/AGENT.md. Delta run.
Load knowledge-graph.json. Diff against git since last session.
Re-descend into changed files and dependents only.
Update scores, flags, and recommendations for affected modules.
```

2. Multi-Persona Descent

What it does: Instead of one generic analysis pass, the descent runs through three expert lenses sequentially. Each persona has different judgment criteria, cares about different things, and produces its own flags.

The three personas:

The Architect (structural integrity)

```
Cares about:
- Separation of concerns (business logic in /lib/, not in components)
- Data flow clarity (can you trace a request from entry to database and back?)
- Dependency direction (do modules depend inward, not in circles?)
- Change safety (if I modify this file, how many things might break?)
- Scalability patterns (will this approach survive 10x users?)
- Convention consistency (does the code follow its own rules?)
```

```
Scoring weight: Code Quality × 2, Test Coverage × 1.5
Flags format: [ARCH] description
```

The User (product experience)

Cares about:

- First-time user experience (what does a new user see? is it obvious what to do?)
- UX state completeness (loading, error, empty, success, disabled states)
- Error message quality (does the user understand what went wrong and what to do?)
- Flow continuity (can the user complete the task without getting lost or stuck?)
- Responsive layout (does it work on the devices the ICP actually uses?)
- Accessibility basics (keyboard navigation, contrast, screen reader hints)
- Copy quality (do labels, buttons, and messages make sense to a non-technical user?)

Scoring weight: Completeness $\times$ 2, Robustness $\times$ 2

Flags format: [USER] description

The Adversary (security and resilience)

Cares about:

- Auth bypass (can I access routes/data I shouldn't?)
- Input manipulation (what if I send malformed data? huge data? missing fields?)
- Payment manipulation (can I change prices client-side? book without paying?)
- Race conditions (can two users book the same slot? double-charge?)
- Data leakage (does the API return fields the client shouldn't see?)
- Privilege escalation (can a customer access consultant admin functions?)
- Webhook spoofing (can I fake a Stripe webhook and get free services?)
- Session handling (what happens with expired/stolen tokens?)

Scoring weight: Security $\times$ 3

Flags format: [ADV] description

How it changes the descent: At each descent level, instead of one MAP $\rightarrow$ SCORE $\rightarrow$ FLAGS pass, the agent runs:

```
MAP (once – structural, shared across personas)
↓
ARCHITECT LENS → architect flags + architect score adjustments
USER LENS → user flags + user score adjustments
ADVERSARY LENS → adversary flags + adversary score adjustments
↓
COMBINED SCORE (weighted merge of all three)
↓
DESCENT DECISION (any persona scored ≤ 5 → mandatory descent)
```

Cross-persona promotion rule: If 2+ personas flag the same file or function → auto-promote to P1 minimum. If all 3 flag it → auto-promote to P0.

This addresses V1's biggest weakness: a single-voice agent that skews toward whatever the LLM "feels like" focusing on. Three explicit personas ensure coverage across structure, experience, and security every single time.

Persona flags stored in knowledge-graph.json (see structure above) so delta runs can compare: "Last session the Architect flagged this file but the User didn't — now after changes, does the User persona see issues too?"

3. Business Impact Scoring

What it does: Every task in recommendations.md gets a **Business Impact** field that connects the code-level issue to a business-level outcome.

Impact scoring matrix:

Impact Level	Definition	Example
Revenue Blocking	Core transaction cannot complete. \$0 until fixed.	Stripe payment intent creation fails silently
Revenue Degrading	Transaction completes but trust/conversion is damaged.	No confirmation email — user thinks booking didn't work
Milestone Blocking	Specific milestone cannot pass its blocker test.	Dashboard shows "undefined" — can't demo to dealers
Milestone Degrading	Milestone achievable but with visible quality issues.	Empty states are blank instead of showing guidance
Operational Risk	System works but failure is likely under real usage.	No webhook idempotency — double charges possible at scale
Technical Debt	No current business impact but compounds over time.	Business logic in 5 components instead of /lib/

How it changes the task format:

markdown

P0-01: Fix silent Stripe payment intent failure

****Business Impact:**** Revenue Blocking

****Impact detail:**** When payment intent creation fails (invalid amount, Stripe outage, network error), the booking form shows a generic "something went wrong" with no retry option. The user cannot complete the core transaction. Every failed payment is a lost booking until this is fixed.

****Estimated conversion impact:**** Any user who hits this abandons. At beta scale (5-10 users/week), this could mean 1-3 lost bookings per week.

Why it matters: V1 recommendations are developer task lists. V2.0 recommendations are business cases. You can show `recommendations.md` to a co-founder, advisor, or investor and they'll understand why task #3 matters more than task #7 — not because of code quality, but because of revenue impact. For a solo founder, this forces you to think like a CEO when prioritizing, not just an engineer.

Integration with judgment phase: The Triple Validation gains a fourth check:

Check 4 – Business Impact Justification:

Can you articulate the business consequence if this stays unfixed?

- Revenue/milestone blocking → P0 candidate
- Revenue/milestone degrading → P1 candidate
- Operational risk → P1/P2 candidate
- Technical debt only → P2/P3 candidate
- Cannot articulate business impact → P3 maximum, regardless of code severity

4. Documentation Drift Detection

What it does: During the ascent phase, the agent cross-references every assertion in documentation against the actual code.

What it checks:

```
## Documentation Drift Report

### Source: CLAUDE.md
| Assertion | File/Section | Code Reality | Drift? |
|-----|-----|-----|-----|
| "API responses use { success, data, error }" | Conventions | 4 routes return different format | ⚠️ DRIFT |
| "Import order: React → third-party → @/lib" | Conventions | 89% compliance, 11% violations | ⚠️ MINOR |
| "Tests next to source: foo.ts → foo.test.ts" | Conventions | 60% of tested files follow this | ⚠️ DRIFT |

### Source: docs/modules/booking-scheduling.md
| Assertion | Code Reality | Drift? |
|-----|-----|-----|
| "Availability checked via is_time_slot_available()" | Booking creation also has inline availability check | ⚠️ DRIFT – duplicate logic |
| "Bookings check availability before creation" | True for API route, not for direct Supabase calls | ⚠️ PARTIAL |

### Source: docs/conventions/API.md
[same structure]
```

Why it matters: Documentation drift is the silent killer of mid-stage projects. Your CLAUDE.md says one thing, the code does another, and every new contributor (including your Claude Code sessions) gets confused. The agent already reads both docs and code — this just makes the comparison explicit.

Drift tasks are auto-generated: For each drift finding, the agent creates a task:

- If the code is right and the doc is wrong → "Update [doc] to reflect [actual behavior]"
 - If the doc is right and the code is wrong → "Fix [file] to follow [convention]"
 - If both are partially right → flag for manual decision
-

V2.0 Summary

Capability	V1	V2.0
Full re-traversal every session	Yes	No — delta analysis via knowledge graph
Analysis perspective	Single generic voice	Three expert personas (Architect, User, Adversary)
Task justification	Effort + priority	Effort + priority + business impact with revenue framing
Documentation accuracy	Reads docs, trusts them	Cross-references docs against code, flags drift
Weekly time commitment	3–5h per cycle	3–5h first run, then 30–60min delta runs
Output audience	Developer only	Developer + business stakeholders

Build time estimate: 2–3 focused sessions to implement and test.

V2.1 — "Product Intelligence Layer"

Prerequisite: V2.0 running and stable with at least 2 full cycles completed.

The jump from V2.0: The agent stops being purely reactive (analyzing what exists) and becomes proactive (telling you what's missing from a product and market perspective). It simulates actual users walking through your app, studies your competitors, and generates architecture decision records that explain *why* changes matter.

What's new

5. User Journey Simulation

What it does: During the ascent phase, after the core transaction trace, the agent simulates complete user journeys for each persona type defined in PROJECT-CONTEXT.

For each user role (e.g., customer, dealer, provider):


```
## User Journey: [Role] – First-Time Experience

### Entry: How does this user find and arrive at the product?
- Landing page exists: Yes / No
- Value proposition visible in < 5 seconds: Yes / No
- CTA clear: Yes / No
- Notes: [what a new user actually sees]

### Onboarding: What happens after signup?
- Signup flow exists: Yes / No
- Steps to first value: [n]
- Empty state on first dashboard: [what does the user see?]
- Guided tour / onboarding prompts: Yes / No
- Can reach core transaction within [n] clicks
- Drop-off risk points: [where might the user give up?]

### Core Loop: What does the repeated usage look like?
- Primary action: [what the user does most often]
- Feedback after action: [what the user sees – confirmation? progress? nothing?]
- Return trigger: [why would the user come back tomorrow?]
- Notification/email bringing them back: Yes / No

### Failure Paths: What happens when things go wrong?
- Error messages: [helpful or cryptic?]
- Recovery options: [can the user fix it themselves?]
- Support path: [can the user get help?]
- Abandonment risk: [where does the user give up and not come back?]

### Journey Score: [X/10]
### Biggest drop-off risk: [one sentence]
### Single highest-impact improvement: [one sentence]
```

How it differs from the User persona in V2.0: The User persona in V2.0 looks at individual components ("does this form have an error state?"). User Journey Simulation in V2.1 looks at the **sequence** ("what does a new dealer experience from Google search to first booking?"). It catches gaps between screens that component-level analysis misses — like "the signup flow works perfectly but dumps you on an empty dashboard with no guidance."

Integration with recommendations: Journey findings generate tasks tagged **[JOURNEY]** with a new priority modifier:

- If a journey step blocks the user from reaching the core transaction → P0 regardless of code quality
- If a journey step confuses or frustrates but doesn't block → P1

- If a journey step is suboptimal but functional → P2

6. Competitive Gap Analysis

What it does: During Phase 0 (Context Ingestion), after reading PROJECT-CONTEXT, the agent:

1. Web-searches each competitor listed in PROJECT-CONTEXT §2
2. Reads their marketing site, feature pages, pricing pages
3. Builds a competitive feature matrix
4. During the descent, flags features competitors offer that your product doesn't

```
## Competitive Feature Matrix
```

Feature	Your Product	Competitor A	Competitor B	Manual Alternative
Online booking	⚠️ Partial	✅ Full	✅ Full	Phone/email
Recurring appointments	❌ Missing	✅	❌	Spreadsheet
Payment processing	⚠️ In progress	✅ Stripe	✅ Square	Invoice
Provider reviews	❌ Missing	✅	✅	Word of mouth
Mobile app	❌	✅	❌	N/A
Video consultations	✅ Daily.co	❌	✅ Zoom	Zoom/Teams

```
## Competitive Gaps Relevant to Current Milestones
```

```
[Only gaps that matter for M1/M2/M3 – not a wishlist of everything competitors do]
```

```
### Gap: [Feature name]
```

- Competitors offering it: [list]
- ICP expectation: [do your target customers expect this?]
- Milestone relevance: [which milestone, if any]
- Build effort estimate: [XS/S/M/L/XL]
- Recommendation: [Build now / Build later / Differentiate differently / Ignore]

Constraint: The agent does NOT recommend building every feature competitors have. It filters through the milestone gates. If a competitive feature doesn't block M1, M2, or M3, it goes to P3 at most. The purpose is strategic awareness, not feature parity panic.

Web search protocol:

- Search each competitor's name + "features" / "pricing" / "reviews"
- Read the top 2–3 results per competitor

- Look specifically for: feature lists, pricing tiers, customer testimonials (reveal pain points), integration lists
 - Time-box: 15 minutes maximum for all competitors combined
 - Cache results in `.agent/competitive-intel.md` — only refresh monthly or on demand
-

7. Architecture Decision Records (ADRs)

What it does: When the agent finds a structural problem that requires a design decision (not just a bug fix), it generates an ADR in `docs/decisions/` instead of (or in addition to) a regular task.

ADR format:

markdown

```
# ADR-[NUMBER]: [Title]
**Date:** [DATE]
**Status:** Proposed (by agent – awaiting developer decision)
**Context generated by:** Recursive Descent Agent v2.1

## Context
[What the agent found during analysis. Reference specific files and findings.]

## Problem
[Why this is a decision, not just a fix. What are the competing concerns?]

## Options

### Option A: [Name]
- Approach: [what to do]
- Effort: [estimate]
- Pros: [list]
- Cons: [list]
- Risk: [what could go wrong]

### Option B: [Name]
[same structure]

### Option C: Do Nothing
- Risk: [what happens if you don't address this]
- Timeline: [how long before this becomes painful]

## Agent Recommendation
[Which option and why, given the project's stage, team, and milestones]

## Decision
[BLANK – developer fills this in]

## Consequences
[What changes if the recommendation is followed – files affected, patterns that]
```

When to generate an ADR vs. a regular task:

- Regular task: "Add error boundary to BookingForm" (clear fix, no design decision)
- ADR: "Redux stores server state that should be fetched fresh — but migrating to React Query touches 15 components" (design decision with tradeoffs)

Why it matters: For a solo developer with offshore contractors, ADRs are the difference between "refactor the state management" (contractor has no idea what you want) and a

structured document with options, tradeoffs, and a clear recommendation they can execute against.

V2.1 Summary

Capability	V2.0	V2.1
Analysis scope	Code + documentation	Code + documentation + user experience + market
User understanding	Component-level UX states	Full journey simulation per user role
Market awareness	None	Competitive feature matrix with milestone-filtered gaps
Structural recommendations	Tasks with steps	Tasks for fixes, ADRs for design decisions
Output audience	Developer + business stakeholders	Developer + business stakeholders + contractors

Build time estimate: 2–3 focused sessions. Competitive analysis needs web search to work in Claude Code (it does). ADR generation is mostly a new output template. User journey simulation is the most complex — it requires the agent to reason about screen sequences, not just individual components.

When to build: After V2.0 has run 3–4 weekly cycles and you trust the scoring system. V2.1 adds layers on top of stable V2.0 outputs.

V2.2 — "The Learning Agent"

Prerequisite: V2.1 running with at least 4–6 weeks of session history and scorecard data.

The jump from V2.1: The agent stops making the same quality of recommendations forever and starts getting better. It tracks what happened to its past recommendations, calibrates its judgment, generates E2E tests that directly validate milestones, and plans its own sessions based on what will produce the most value today.

What's new

8. Outcome Tracking + Judgment Calibration

What it does: At the start of every session (during pre-flight), the agent reviews its prior recommendations and checks what actually happened:

```
## Outcome Tracking – [DATE]
```

```
### Recommendations from [prior session date]
```

```
| Task ID | Priority | Status | Outcome | Judgment Accuracy |
|-----|-----|-----|-----|-----|
| P0-01 | P0 | ✅ Implemented (commit abc123) | Module score: 4→7 | ✅
Accurate – was truly blocking |
| P0-02 | P0 | ❌ Still open (3 weeks) | No consequence yet | ⚠️ Overrated?
– not actually blocking |
| P1-03 | P1 | ✅ Implemented | Module score: unchanged | ❌ Low impact –
should have been P2 |
| P1-05 | P1 | 🔄 Partially done | – | – Pending |
| P2-08 | P2 | ❌ Ignored | – | ⓘ Expected for P2 |
```

How it checks:

1. `git log --oneline --since="[last session]"` → find commits that reference task IDs
2. For implemented tasks: compare the module's score before and after
3. For open tasks: check how long they've been open and whether the predicted consequence materialized

Judgment calibration file: `.agent/judgment-calibration.md`

```

## Calibration – Updated [DATE]

### Priority Accuracy (last 20 tasks)
- P0 tasks that were truly blocking: [n]/[total P0s] ([%])
- P1 tasks that had measurable impact when fixed: [n]/[total P1s] ([%])
- P2 tasks that were picked up: [n]/[total P2s] ([%])
- Average overrating: [P0s that should have been P1/P2: n]
- Average underrating: [P1/P2s that turned out to be P0: n]

### Focus Area Accuracy
- Booking/Payment findings: [% that led to score improvement when fixed]
- UX State findings: [%]
- Component Structure findings: [%]
- Security findings: [%]

### Persona Accuracy
- Architect flags that led to real improvements: [%]
- User flags that led to real improvements: [%]
- Adversary flags that led to real improvements: [%]

### Calibration Adjustments
Based on the above data, adjust future judgment:
- [adjustment 1: e.g., "Architect persona overrates component structure refactors – downweight by 1 priority level unless on critical path"]
- [adjustment 2: e.g., "P0 threshold too loose for UX findings – require Revenue Blocking or Milestone Blocking impact, not just Degrading"]
- [adjustment 3]

```

Why it matters: Every recommendation system has bias. Without feedback, the agent will perpetually over-recommend certain types of work (probably refactors, which LLMs love suggesting) and under-recommend others. Outcome tracking is the closed loop that makes the agent's judgment actually improve over time instead of staying static.

9. Milestone-Driven E2E Test Generation

What it does: For each milestone's blocker test (from PROJECT-CONTEXT §3), the agent generates end-to-end test scenarios and writes executable test files.

```
## Milestone 1 Blocker Test: "Can you screen-share the app for 10 minutes without hitting an error?"
```

```
### E2E Test Scenarios Generated:
```

```
#### Test 1: New dealer visits homepage and navigates to provider listing
```

```
- Steps:
```

1. Navigate to /
2. Assert: hero section visible, no "undefined" or broken images
3. Click "Find a Provider"
4. Assert: provider listing loads within 3 seconds
5. Assert: at least 1 provider card rendered with name, image, rating
6. Assert: no console errors

```
#### Test 2: Dealer views a provider profile
```

```
- Steps:
```

1. Navigate to /providers/[test-provider-slug]
2. Assert: profile photo, name, services, availability visible
3. Assert: "Book Now" CTA visible and clickable
4. Assert: no layout shifts, no blank sections
5. Assert: mobile viewport renders correctly

```
#### Test 3: Dealer starts a booking (demo mode – no payment)
```

```
[...]
```

The agent then writes the actual test file:

```
typescript
```

```
// tests/e2e/milestone-1-demo-ready.spec.ts
// Generated by Recursive Descent Agent v2.2
// Validates: Milestone 1 – Demo-Ready MVP

import { test, expect } from '@playwright/test';

test.describe('Milestone 1: Demo-Ready MVP', () => {
  test('new dealer can browse providers without errors', async ({ page }) => {
    // ...
  });
});
```

Why it matters: V1 writes unit tests for code gaps. V2.0 writes unit tests for code gaps. V2.2 writes E2E tests for *milestone readiness*. This is a fundamentally different value — instead of "this function is tested," it's "this milestone will pass its blocker test." You can run the E2E suite before a demo and know in 2 minutes whether you're ready.

Integration with health scorecard: Milestone readiness % is no longer estimated — it's measured by the pass rate of milestone E2E tests:

Milestone 1 readiness: 72% (8/11 E2E tests passing)

Failing: test-3 (booking form timeout), test-7 (dashboard empty state), test-9 (mobile nav)

10. Session Planning Intelligence

What it does: At the start of each session, instead of the developer choosing a session format, the agent analyzes the current state and recommends the optimal session plan.

```
## Recommended Session Plan – [DATE]
```

```
### Available time: [developer specifies, e.g., "90 minutes"]
```

```
### Current state analysis:
```

- Last session: [date, what was done]
- Scorecard: [overall health, trend]
- Open P0s: [n] – oldest: [age in days]
- Open P1s: [n]
- Modules that regressed since last session: [list]
- Milestone E2E pass rate: M1 [%], M2 [%], M3 [%]
- Files changed since last session: [n]

```
### Recommended plan for 90 minutes:
```

```
**Option A – Highest Impact (recommended)**
```

1. Delta run on [n] changed files (est. 20 min)
 - Reason: PR merged yesterday touching payment module, which scored 5 last week
2. Action phase: write E2E test for milestone 1 blocker test #3 (est. 25 min)
 - Reason: M1 readiness stuck at 72%, this test covers the booking form gap
3. Action phase: fix P0-04 (missing Stripe error handling) (est. 30 min)
 - Reason: oldest P0, revenue-blocking, 15 lines of code
4. Re-score payment and booking modules (est. 15 min)

```
**Option B – Broad Progress**
```

1. Full re-score of all modules (est. 20 min)
2. Re-run cross-cut analysis (est. 25 min)
3. Regenerate recommendations from updated data (est. 20 min)
4. Action phase: fix 2 smallest P1s in booking cluster (est. 25 min)

```
**Option C – Deep Dive**
```

1. Full descent Level 2-3 into auth module (est. 60 min)
 - Reason: auth hasn't been analyzed since [date], 3 files changed
2. Update scorecard (est. 15 min)
3. Generate ADR for auth refactoring decision (est. 15 min)

How it decides:

Priority 1: Any module that regressed since last session (investigate immediately)

Priority 2: Oldest open PO that can be fixed within available time

Priority 3: Milestone with lowest E2E pass rate (focus tests and fixes there)

Priority 4: Delta analysis on changed files (maintain knowledge graph freshness)

Priority 5: Re-scoring and cross-cut refresh (keep the data accurate)

Priority 6: New descent into unanalyzed or stale modules

Developer override: The plan is a recommendation, not a mandate. The developer can always say:

Read .agent/AGENT.md. Ignore session plan. Execute [specific command].

But the plan ensures that if you sit down for your daily 1-2 hours and don't know where to start, the agent tells you what will produce the most value today based on everything it knows about the project's current state.

V2.2 Summary

Capability	V2.1	V2.2
Recommendation quality	Static (same calibration forever)	Self-improving (tracks outcomes, adjusts judgment)
Test generation	Unit tests for code gaps	Unit tests + E2E tests for milestone readiness
Milestone measurement	Estimated %	Measured via E2E pass rate
Session planning	Developer chooses from menu	Agent recommends optimal plan for available time
Long-term trajectory	Repeating analysis tool	Learning system that gets more accurate weekly

Build time estimate: 3-4 focused sessions. Outcome tracking requires git log parsing and score comparison logic. E2E test generation requires understanding the project's test framework and writing executable code. Session planning requires reasoning across all agent state files.

When to build: After V2.1 has produced 4-6 weeks of data. Outcome tracking needs history to calibrate against. E2E generation needs stable user journey maps from V2.1. Session

planning needs enough scorecard history to identify trends and regressions.

Full Version Comparison

	V1.0	V2.0	V2.1	V2.2
Re-traversal	Full every time	Delta via knowledge graph	Delta	Delta
Analysis lens	Single voice	Three personas	Three personas	Three personas (calibrated)
Task justification	Effort + priority	+ Business impact	+ Business impact	+ Outcome-validated impact
Documentation	Reads and trusts	Cross-references for drift	+ ADRs for decisions	+ ADRs
Market awareness	None	None	Competitive gap analysis	Competitive gap analysis
User understanding	Component UX states	Component UX states	+ Full journey simulation	+ Journey simulation
Test output	None	Unit tests for gaps	Unit tests	+ E2E for milestones
Milestone tracking	Estimated %	Estimated %	Estimated %	Measured via E2E pass rate
Self-improvement	None	None	None	Outcome tracking + calibration
Session planning	Manual selection	Manual selection	Manual selection	Agent-recommended
Weekly time (after first run)	3-5h	30-60min	45-75min	30-60min (better targeted)

Implementation Order

Week 1-2:	Build V2.0 (persistent memory + personas + impact + drift)
Week 3-4:	Run V2.0 on Field Advisor, stabilize, fix edge cases
Week 5-6:	Build V2.1 (journeys + competitive + ADRs)
Week 7-8:	Run V2.1, accumulate 2+ weeks of data
Week 9-10:	Build V2.2 (outcomes + E2E + session planning)
Week 11+:	V2.2 running, self-improving weekly

Each version is fully usable on its own. You don't need V2.2 to get value from V2.0.

The compounding happens naturally: V2.0 builds the data → V2.1 adds product context → V2.2 learns from all of it.